

LabReSiD26 – Hands-On 6

Davide Ferrara – 518629

1 Esercizio 1: Analisi di server_epoll.c

Domanda 1 – Differenza strutturale tra server_epoll.c e server_select_demo.c

La differenza strutturale più evidente è l'eliminazione della rubrica `client_fds[]`, di `max_fd` e della ricostruzione del set ad ogni ciclo con `FD_ZERO/FD_SET`. In `server_select_demo.c` il programma deve mantenere queste strutture manualmente; in `server_epoll_demo.c` è il kernel a tracciare i fd registrati tramite il suo albero rosso-nero interno. `max_fd` non è più necessario perché `epoll_wait()` non scansiona un range di fd: restituisce direttamente i soli fd pronti nell'array `events[]`.

Prima (select):

```
1 /* rubrica da mantenere manualmente */
2 int client_fds[MAX_CLIENTS];
3 for (int i = 0; i < MAX_CLIENTS; i++) client_fds[i] =
    -1;
4 int max_fd = server_fd;
5
6 /* ad ogni iterazione si ricostruisce il set */
7 fd_set read_set;
8 FD_ZERO(&read_set);
9 FD_SET(server_fd, &read_set);
10 for (int i = 0; i < MAX_CLIENTS; i++)
11     if (client_fds[i] != -1) FD_SET(client_fds[i], &
        read_set);
12 int n = select(max_fd + 1, &read_set, NULL, NULL, NULL);
```

Dopo (epoll):

```
1 /* nessuna rubrica, nessun max_fd */
2 int epfd = epoll_create1(0);
3 struct epoll_event events[MAX_EVENTS];
```

```

4
5  /* ad ogni iterazione: epoll_wait restituisce solo i fd
   pronte */
6  int n = epoll_wait(epfd, events, MAX_EVENTS, -1);
7  for (int i = 0; i < n; i++) {
8      int fd = events[i].data.fd;
9  }

```

Domanda 2 – Dove sono finiti `client_fds[]` e `max_fd`

Entrambe le strutture spariscono dal codice utente. Se ne occupa il kernel: ogni `epoll_ctl(EPOLL_CTL_ADD, fd, ...)` inserisce il fd nell'albero rosso-nero interno dell'istanza `epoll`.

`epoll_wait()` si limita a copiare i fd pronti nell'array `events[]` con costo $O(\text{eventi pronti})$.

`max_fd` non esiste più perché `epoll_wait()` non riceve un range di fd da scansionare ma conosce già i fd registrati tramite l'albero rosso-nero e consegna direttamente ciò che è pronto.

Domanda 3 – `epoll_wait()` ritorna con $N > 1$

Dal manuale:

```

On success, epoll_wait() returns the number of file
descriptors ready for the requested I/O operation,
or zero if no file descriptor became ready
during the requested timeout milliseconds.

```

`epoll_wait()` può quindi ritornare con $n > 1$ quando più fd diventano pronti prima che il server torni a chiamarlo. Nel caso dello script fornito per aumentare la probabilità che due o più client si sovrappongano è necessario diminuire il delay tramite il secondo parametro dello script:

```

1 ./test_multifd.sh 10 0

```

Con `delay=0` le connessioni avvengono immediatamente e con un numero alto di client aumenta la probabilità che più eventi si accodino nella ready list prima della prossima chiamata a `epoll_wait()`, ottenendo $n > 1$.

Domanda 4 – `epoll_ctl(DEL)` prima o dopo `close(fd)`

`epoll_ctl(DEL)` va chiamato **prima** di `close(fd)`. Dal kernel 2.6.9 la rimozione esplicita è diventata opzionale (il kernel rimuove automaticamente il fd alla chiusura), ma l'ordine corretto resta DEL poi close per due motivi.

Gestione degli errori. Dal manuale: “*When successful, `epoll_ctl()` returns zero. When an error occurs, `epoll_ctl()` returns -1 and `errno` is set to indicate the error.*” Se si chiude prima il fd, questo non è più valido: una successiva `epoll_ctl(DEL)` fallirebbe con EBADF (Bad File Descriptr) e non sarebbe possibile gestire l'errore.

Rischio di fd reuse. Alla `close()`, il kernel libera immediatamente il numero di fd. Un altro thread o la stessa `accept()` potrebbe ricevere quel numero come nuovo fd. Chiamare `epoll_ctl(DEL, fd)` dopo rimuoverebbe il fd *nuovo* dall'istanza epoll, non quello vecchio, causando un comportamento indefinito silenzioso senza alcun errore visibile.

L'ordine corretto è quindi:

```
1 epoll_ctl(epfd, EPOLL_CTL_DEL, fd, NULL);  
2 close(fd);
```

Domanda 5 – `epoll_create1()` chiamato una volta sola

`epoll_create1()` crea un'istanza epoll nel kernel e restituisce un file descriptor che la rappresenta (`epfd`). Chiamarlo ad ogni iterazione del loop causa tre problemi.

File descriptor leak. Ogni chiamata apre un nuovo `epfd` senza che il precedente venga mai chiuso. Il kernel mantiene allocati albero rosso-nero e ready list per ogni istanza: iterando abbastanza a lungo si esauriscono i fd disponibili del processo (solitamente 1024 massimi per processo).

```
1 ulimit -n  
2 1024 /* Numero massimo di FD nel mio sistema*/
```

Memoria kernel non liberata. Le strutture interne di ogni istanza epoll (albero rosso-nero + ready list) rimangono allocate fino alla `close(epfd)`. Con un nuovo `epfd` ad ogni ciclo e nessuna chiusura, la memoria kernel cresce senza limite causando possibili OOM.

Perdita delle registrazioni. Ogni nuova istanza parte vuota: tutti i client già connessi e registrati con `epoll_ctl(EPOLL_CTL_ADD)` spariscono. Il server perderebbe traccia di tutte le connessioni attive ad ogni iterazione.

Domanda 6 – Confronto riga di compilazione: epoll vs io_uring

Un programma che usa `io_uring` tramite l'API ad alto livello richiede **liburing** (`-luring`), la libreria userspace di Jens Axboe.

Un esempio reale verificato è **QEMU**: il suo `meson.build`¹ dichiara esplicitamente la dipendenza da `liburing` tramite `pkg-config`:

```
[meson.build]
linux_io_uring = dependency('liburing', version: '>=0.3',
                             required: get_option('linux_io_uring'),
                             method: 'pkg-config')
```

Questo produce in compilazione il flag `-luring`.

Confronto righe di compilazione:

```
1 # server_epoll.c con nessuna libreria esterna
2 gcc -Wall -O2 -o server_epoll server_epoll.c
3
4 # programma con link di uring
5 gcc -Wall -O2 -o server_uring server_uring.c -luring
```

¹<https://github.com/qemu/qemu/blob/master/meson.build>